

```

/**
 * server/lib/prompt-injection-guard.ts (hardened v2)
 *
 * Fixes applied:
 * - Gaming context checked in 160-char window around the match (not full string).
 * - Critical patterns always block; warning patterns NEVER block – only score/log.
 * - Gaming regex written as direct literal (not escaped through .replace which
 *   was turning "assassin.?s creed" into a literal dot-question-mark).
 * - SQL injection checks path + query + body slice, not just req.path.
 * - IP allowlist loaded from DB with hardcoded fallback.
 * - req.ip trust left to caller.
 */

import { db }          from '../db.js';
import { sql }         from 'drizzle-orm';
import { createLogger } from './logger.js';

// log is used in isTrustedIp and checkSqlInjection for allowlist refresh failures
const log = createLogger('prompt-injection-guard');

// — Gaming context (checked in local window, not full string) —————
// Fix #3: new RegExp([...].join('|'), 'i') – immune to accidental line breaks
// that would silently break a regex literal when editors wrap long lines.

const GAMING_TERMS_PATTERN = new RegExp([
  "assassin'?s creed",
  "assassins creed",
  "valhalla",
  "dragon age",
  "veilguard",
  "battlefield",
  "bf6",
  "ps5",
  "playstation",
  "xbox",
  "stealth",
  "assassin",
  "exploit",
  "glitch",
  "speedrun",
  "boss fight",
  "highlight",
  "montage",
  "walkthrough",
  "playthrough",
  "gameplay",
  "gaming",
  "no commentary",

```

```

    "et gaming",
  ].join("|"), "i");

const CONTEXT_WINDOW = 160; // chars before + after the match to search for gaming terms

function hasGamingContextNearMatch(input: string, matchIndex: number): boolean {
  const start = Math.max(0, matchIndex - CONTEXT_WINDOW);
  const end = Math.min(input.length, matchIndex + CONTEXT_WINDOW);
  return GAMING_TERMS_PATTERN.test(input.slice(start, end));
}

// — Patterns —————

type PatternDef = {
  pattern:      RegExp;
  label:        string;
  alwaysBlock: boolean; // if true, blocks even with gaming context
  severity:     'critical' | 'warning';
};

const PATTERNS: PatternDef[] = [
  // CRITICAL – never gaming-safe
  { pattern: /ignore\s+(all\s+)?(previous|prior)\s+instructions/i, label:
'ignore_override', alwaysBlock: true, severity: 'critical' },
  { pattern: /system\s+prompt|<\s*system\s*>/i, label:
'system_prompt_leak', alwaysBlock: true, severity: 'critical' },
  { pattern: /jailbreak|DAN\s+mode|developer\s+mode/i, label:
'jailbreak_attempt', alwaysBlock: true, severity: 'critical' },
  { pattern: /\[INST\]|\[\[INST\]|<\[im_start\]>/i, label:
'prompt_injection', alwaysBlock: true, severity: 'critical' },
  { pattern: /disregard\s+(your\s+)?(previous|prior|all)\s+(instructions|training)/i, label:
'training_override', alwaysBlock: true, severity: 'critical' },
  // WARNING – gaming-safe when gaming context detected near the match
  { pattern: /you\s+are\s+(now|a|an)\s+(?!creator|content)/i, label:
'persona_injection', alwaysBlock: false, severity: 'warning' },
  { pattern: /act\s+as\s+(if\s+you\s+are|a|an)\s+(?!my|the|a\s+creator)/i, label:
'roleplay_injection', alwaysBlock: false, severity: 'warning' },
  { pattern: /bypass\s+(your|the|all)?\s*(filter|safety|guard|restrict)/i, label:
'filter_bypass', alwaysBlock: false, severity: 'warning' },
];

// — IP allowlist (DB-backed with hardcoded fallback) —————

const HARDCODED_TRUSTED_PREFIXES = [
  '35.191.', // GCP LB health checks
  '130.211.', // GCP LB health checks
  '209.85.', // Google crawlers
  '66.249.', // Googlebot
  '127.', // localhost
  '10.', // private RFC 1918

```

```

'192.168.', // private RFC 1918
'::1',      // IPv6 localhost
];

let dbAllowlistPrefixes: string[] = [];
let allowlistLoadedAt = 0;
const ALLOWLIST_TTL_MS = 5 * 60 * 1000; // reload from DB every 5 min

async function isTrustedIp(ip: string): Promise<boolean> {
  // Refresh from DB periodically
  if (Date.now() - allowlistLoadedAt > ALLOWLIST_TTL_MS) {
    try {
      const rows = await db.execute<{ ip_prefix: string }>(
        sql`SELECT ip_prefix FROM security_ip_allowlist`
      );
      dbAllowlistPrefixes = rows.rows.map(r => r.ip_prefix);
      allowlistLoadedAt = Date.now();
    } catch (err) {
      // Fix #4: stamp the timestamp even on failure so we don't retry the DB
      // on every single request when the table is missing or DB is unavailable.
      allowlistLoadedAt = Date.now();
      log.debug('[InjectionGuard] DB allowlist unavailable – using hardcoded fallback');
    }
  }

  const prefixes = dbAllowlistPrefixes.length > 0
    ? dbAllowlistPrefixes
    : HARDCODED_TRUSTED_PREFIXES;

  return prefixes.some(prefix => ip.startsWith(prefix));
}

// ——— Public API ———

export interface InjectionResult {
  blocked:          boolean;
  severity:         'critical' | 'warning' | 'safe';
  label:            string | null;
  gamingFalsePositive: boolean;
}

/**
 * Check a string for prompt injection.
 *
 * @param input - the content to check (AI-generated text, user input, etc.)
 * @returns InjectionResult
 *
 * Usage in middleware:
 *
 * const result = checkPromptInjection(input);
 * if (result.blocked) {

```

```

*   if (!result.gamingFalsePositive) {
*       logSecurityEvent(result.label, result.severity);
*       if (result.severity === 'critical') return res.status(400).end();
*   } else {
*       log.debug(`[Security] Gaming false positive suppressed: ${result.label}`);
*   }
* }
*/

export function checkPromptInjection(input: string): InjectionResult {
  if (!input || input.trim().length === 0) {
    return { blocked: false, severity: 'safe', label: null, gamingFalsePositive: false };
  }

  for (const { pattern, label, alwaysBlock, severity } of PATTERNS) {
    const match = pattern.exec(input);
    if (!match) continue;

    if (alwaysBlock) {
      return { blocked: true, severity, label, gamingFalsePositive: false };
    }

    // Warning-level: check gaming context in local window around the match.
    // Fix #4: warning patterns NEVER block – they only score/log.
    // Return blocked:false with severity:'warning' so middleware can decide.
    if (hasGamingContextNearMatch(input, match.index)) {
      // Gaming content near the pattern – suppress even the warning log
      return { blocked: false, severity: 'safe', label, gamingFalsePositive: true };
    }

    // No gaming context nearby – flag as warning but still DO NOT block
    return { blocked: false, severity: 'warning', label, gamingFalsePositive: false };
  }

  return { blocked: false, severity: 'safe', label: null, gamingFalsePositive: false };
}

/**
 * Check a request for SQL injection.
 * Caller must pass the resolved client IP (respecting trust proxy config).
 *
 * Fix #5: input should be path + query + bounded body slice, NOT just path.
 * Build the input string in middleware like this:
 *
 *   const sqlInput = [
 *     req.path,
 *     JSON.stringify(req.query ?? {}).slice(0, 2000),
 *     JSON.stringify(req.body ?? {}).slice(0, 5000),
 *   ].join(' ');
 *   const result = await checkSqlInjection(sqlInput, clientIp);
 */

```

```
export async function checkSqlInjection(
  input: string,
  clientIp: string,
): Promise<InjectionResult> {
  if (await isTrustedIp(clientIp)) {
    return { blocked: false, severity: 'safe', label: null, gamingFalsePositive: false };
  }

  const SQL_PATTERNS = [
    /(\bor\b|\band\b)\s+\d+=\d+/i,
    /union\s+select/i,
    /;\s*drop\s+table/i,
    /'\s*or\s+'[^']*'\s*=\s*'/i,
    /xp_cmdshell|exec\s*\(/i,
  ];

  for (const p of SQL_PATTERNS) {
    if (p.test(input)) {
      return { blocked: true, severity: 'critical', label: 'sql_injection', gamingFalsePositive:
false };
    }
  }

  return { blocked: false, severity: 'safe', label: null, gamingFalsePositive: false };
}
```